

Birhane Hiwot Sunday School Management System

(ብርሃነ ሕይወት ሰንበት ትምህርት ቤት የአቴንዳንስ እና የትምህርት ክፍል አስተዳደር ሲስተም)

Comprehensive Technical Documentation, Architecture & Domain Guide

Version: 1.0.0
Target Organization: Birhane Hiwot Sunday School (ብርሃነ ሕይወት ሰንበት ትምህርት ቤት)
Maintained By: Development & QA Engineering Team
Last Updated: August 26, 2026

Table of Contents

- 1. System Overview & Objectives
- 2. Technology Stack & Tools
- 3. System Architecture & Data Flow
- 4. Domain Model & Core Business Logic
 - 4.1 Ethiopian Calendar & Date System
 - 4.2 Student Classifications & Canonical Grade Hierarchy
 - 4.3 Role-Based Access Control (RBAC) & Dynamic Permissions
 - 4.4 Student Registration, Gating Windows & Duplicate Detection
 - 4.5 Bulk Attendance Marking & Distributed Locking
 - 4.6 HMAC-SHA256 Signed QR Code Engine
 - 4.7 Sunday School Curriculum, Grouped Subjects & Teacher Assignments
 - 4.8 Student Academic Results & University Letter Grading
 - 4.9 13-Month Ethiopian Fee Payment Tracking
 - 4.10 Student Registration Requests & Approval Workflow
 - 4.11 Offline-First Mobile Architecture & Dexie Sync Queue
- 5. Complete API Route Directory (38 Endpoints)
- 6. Database Schema & MongoDB Collections
- 7. Automated Testing, QA & Tooling
- 8. Scripts, Migrations & Excel Seeders
- 9. Configuration & Environment Variables
- 10. Local Development & Deployment Guide

1. System Overview & Objectives

The **Birhane Hiwot Sunday School Attendance & Education Management System** is a mission-critical web and mobile platform engineered for Ethiopian Orthodox Sunday Schools. It centralizes student demographic records, daily/weekly attendance taking, Orthodox theological curriculum management, academic score evaluations, 13-month Ethiopian fee payments, and distributed facilitator operations across different class classifications.

Key Objectives

- Ethiopian Native Experience:** First-class support for Ge'ez/Amharic typography, Ethiopian calendar calculations (13 months including *Pagumē*), and cultural naming conventions (First Name, Father's Name, Grandfather's Name, Mother's Name, Christian/Baptismal Name).
- Concurrency & High Reliability:** Distributed Redis locks preventing race conditions during attendance submissions and student record allocations.
- Role-Based Isolation:** Tailored workspaces for Super Admins, HR Admins, Education Admins, Attendance Facilitators, Education Facilitators, and Teachers.
- Tamper-Proof Verification:** HMAC-SHA256 signed student QR codes for rapid mobile/camera attendance scanning.
- Offline Resilience:** Dedicated Capacitor/PWA mobile app utilizing local IndexedDB storage with automatic background synchronization.

2. Technology Stack & Tools

Frontend (Web Application)

- Framework:** [Next.js 15.4.10](#) (App Router, Server Components, Route Handlers, Turbopack).
- Core Library:** [React 19.1.0](#).
- Language:** [TypeScript 5.9.2](#).
- Styling & Design System:** [Tailwind CSS v4](#) with custom CSS design tokens, HSL palette, dark mode accents, and smooth micro-animations.
- Icons & UI Primitives:** [@heroicons/react 2.2.0](#), [lucide-react 0.542.0](#), [@radix-ui/react-dialog](#).
- Animations & Transitions:** [framer-motion 12.23.12](#).
- Data Fetching & Caching:** [swr 2.3.6](#) for client-side optimistic UI updates.
- Notifications:** [react-hot-toast 2.6.0](#).

Mobile Application ([attendance-mobile/](#))

- **Runtime:** Hybrid Mobile App powered by [Capacitor](#) & Vite.
- **Local Storage / Offline Database:** [Dexie.js](#) (IndexedDB wrapper) with sync queue transaction management.
- **Camera QR Scanner:** [html5-qrcode](#) 2.3.8 with responsive viewport detection.

Backend & API Layer

- **Authentication:**
 - **Web:** [NextAuth.js v4.24.11](#) using credentials provider, secure HTTP-only cookies, CSRF protection, and JWT session tokens (24h validity).
 - **Mobile:** Custom signed HS256 JWT tokens using [jose](#) library (7-day validity) via [/api/mobile/auth](#) with full CORS preflight support.
- **Password Security:** [bcryptjs](#) 3.0.2 with 10 salt rounds.
- **Distributed Locking & Caching:** [@upstash/redis](#) 1.38.2 with local in-memory mutex fallback.

Database & Storage

- **Primary Database:** [MongoDB Atlas](#) with native [mongodb](#) 6.17.0 driver.
- **Cloud Document Sync:** Google Sheets API v4 via [googleapis](#) 159.0.0.

Document & Media Generation

- **PDF Generation:** [jspdf](#) 3.0.1 + [jspdf-autotable](#) 5.0.2 with custom table formatters.
- **Excel Export/Import:** [xlsx](#) 0.18.5 + [file-saver](#) 2.0.5 + [papaparse](#) 5.5.3.
- **QR Code Engine:** [qrcode](#) 1.5.4 (Base64 data URL + PNG stream generation).

QA & Automation Tools

- **Test Runner:** Native Node.js ESM test suite ([scripts/run-automation-tests.mjs](#)) covering 63 end-to-end test cases with zero external runtime dependencies.
- **API Documentation:** OpenAPI 3.0.3 specification ([public/openapi.json](#)) rendered interactively at [/api-docs](#) (Swagger UI).
- **Postman Assets:** Complete Postman Collection ([Birhane-Hiwot-API.postman_collection.json](#)) and Environment presets.

3. System Architecture & Data Flow



4. Domain Model & Core Business Logic

4.1 Ethiopian Calendar & Date System

The system operates natively with the **Ethiopian Calendar (EC)**:

- 13 Months:** 12 months of 30 days (*Meskerem, Tikimt, Hidar, Tahsas, Tir, Yekatit, Megabit, Miyazia, Ginbot, Sene, Hamle, Nehase*) and 1 epagomenal month of 5 or 6 days (*Pagumē*).
- Academic Years:** Default academic year is represented as Ethiopian year **2018** (corresponding to 2025/2026 Gregorian).
- Calendar Mode Switch:** `ATTENDANCE_CALENDAR_MODE` in `src/lib/constants.ts` supports:
 - `"all_days"`: Enables attendance marking for any day (ideal for testing, makeup sessions, and summer programs).
 - `"sundays_only"`: Restricts attendance date pickers strictly to Ethiopian Sundays.

4.2 Student Classifications & Canonical Grade Hierarchy

Students belong to one of 5 distinct classifications, each mapped to specific grade levels and schedules:

Classification	Amharic Name	Schedule	Grade Options
Regular	መደበኛ	Every Sunday	ቅድመ መደበኛ (Grade 0) up to አስራ ሁለተኛ ክፍል (Grade 12)
Extension	ጭታ / ቅዳሜ	1st Sunday of each Ethiopian month (excl. 13th)	1ኛ ዓመት (Level 1), 2ኛ ዓመት (Level 2)
SignLanguage	የምልክት ቋንቋ	Every Sunday with Sign Language	ምልክት ቋንቋ (Sign Language Class)
Summer	ከረምት	Daily / Summer schedule	ሰባተኛ ክፍል ጥዋት (7-1), ሰባተኛ ክፍል ከሰዓት (7-2)
begena	በገና	6-month specialized program	በገና (BeGena)

Canonical Grade Options

```
export const GRADE_OPTIONS = [
  { value: "ቅድመ መደበኛ", label: "ቅድመ መደበኛ · Grade 0", number: 0 },
  { value: "አንደኛ ክፍል", label: "አንደኛ ክፍል · Grade 1", number: 1 },
  { value: "ሁለተኛ ክፍል", label: "ሁለተኛ ክፍል · Grade 2", number: 2 },
  { value: "ሦስተኛ ክፍል", label: "ሦስተኛ ክፍል · Grade 3", number: 3 },
  { value: "አራተኛ ክፍል", label: "አራተኛ ክፍል · Grade 4", number: 4 },
  { value: "አምስተኛ ክፍል", label: "አምስተኛ ክፍል · Grade 5", number: 5 },
  { value: "ስድስተኛ ክፍል", label: "ስድስተኛ ክፍል · Grade 6", number: 6 },
  { value: "ሰባተኛ ክፍል ጥዋት", label: "ሰባተኛ ክፍል ጥዋት · 7-1", number: 7 },
  { value: "ሰባተኛ ክፍል ከሰዓት", label: "ሰባተኛ ክፍል ከሰዓት · 7-2", number: 7 },
  { value: "ስምንተኛ ክፍል", label: "ስምንተኛ ክፍል · Grade 8", number: 8 },
  { value: "ዘጠነኛ ክፍል", label: "ዘጠነኛ ክፍል · Grade 9", number: 9 },
  { value: "አስረኛ ክፍል", label: "አስረኛ ክፍል · Grade 10", number: 10 },
  { value: "አስራ አንደኛ ክፍል", label: "አስራ አንደኛ ክፍል · Grade 11", number: 11 },
  { value: "አስራ ሁለተኛ ክፍል", label: "አስራ ሁለተኛ ክፍል · Grade 12", number: 12 },
];
```

4.3 Role-Based Access Control (RBAC) & Dynamic Permissions

The system enforces strict multi-tenant department segregation:

Role	Accessible Web Routes	Accessible API Routes	Key Permissions
Super Admin	<code>/super-admin/*</code> , <code>/admin/*</code>	All API endpoints	System bootstrap, department admins management, global category period configuration, student request approvals.
HR Admin	<code>/hr/*</code> , <code>/admin/students</code> , <code>/admin/facilitators</code> , <code>/admin/reports</code>	<code>/api/students</code> , <code>/api/facilitators</code> , <code>/api/attendance</code> , <code>/api/payment</code>	HR facilitator user management, student registry, student profile edits, <code>canAddStudent</code> permission toggles.
Education Admin	<code>/education/*</code> , <code>/admin/facilitators</code> , <code>/admin/reports</code>	<code>/api/education-facilitators</code> , <code>/api/teachers</code> , <code>/api/subjects</code> , <code>/api/student-results</code>	Education facilitators, teacher accounts, curriculum subject definitions, teacher assignments, student exam score entry.
Attendance Facilitator	<code>/facilitator/attendance/*</code>	<code>/api/attendance</code> , <code>/api/students</code> (gated)	Mark attendance for assigned grades; register new students only if <code>canAddStudent === true</code> for their assigned grades.

Role	Accessible Web Routes	Accessible API Routes	Key Permissions
Education Facilitator	/facilitator/results/*, /facilitator/dashboard	/api/student-results, /api/subjects	Enter student exam and assignment scores, view grade rosters.
Teacher	Direct roster view	/api/teacher-assignments, /api/subjects	View assigned courses and enrolled student lists.

Dynamic `canAddStudent` Permission

When an HR Admin or Super Admin toggles `canAddStudent` on a facilitator via `PATCH /api/facilitators/{id}`, the NextAuth JWT callback dynamically refreshes permissions on every request by querying MongoDB `_id`, allowing immediate permission grants **without forcing the facilitator to log out and re-login**.

4.4 Student Registration, Gating Windows & Duplicate Detection

1. Registration Window Gating (`CategoryPeriod`)

Before creating any student record, the server verifies `registrationClosedDate` from the `category_periods` collection for the requested (`classification`, `academicYear`). If `today >= registrationClosedDate`, the API rejects creation with `403 Forbidden` (`Registration is closed for this category`).

2. Name & Identity Duplicate Detection Algorithm

The system executes a case-insensitive, whitespace-trimmed 5-point match against the `students` collection: `$$\text{Duplicate Match} = (\text{First_Name} \land \text{Father_Name} \land \text{Grandfather_Name} \land \text{Mothers_Name} \land \text{Sex})$$`

3. Automatic Unique_ID Generation

Format: `BH-{AcademicYear}-{ClassificationCode}-{SequentialNumber}` (e.g. `BH-2018-001`). Student creation uses a distributed lock (`lock:student:{Unique_ID}`) to prevent ID duplication during rapid concurrent registrations.

4.5 Bulk Attendance Marking & Distributed Locking

Attendance records are stored as individual documents keyed by (`studentId`, `date`):

```
export interface Attendance {
  studentId: string;
  date: string;           // Ethiopian date string e.g. "2018-05-15"
  present: boolean;
  hasPermission: boolean; // Excused absence flag
  reason?: string;        // e.g. "በሀመም ምክንያት"
  markedBy?: string;      // Facilitator email or name
  timestamp?: string;     // Recorded timestamp
}
```

Distributed Concurrency Locking

To prevent race conditions when multiple facilitators mark attendance simultaneously for the same date:

```
await withDistributedLock(`lock:attendance:${date}`, async () => {
  // 1. Bulk upsert operations using MongoDB bulkWrite
  // 2. Aggregate counts
  // 3. Return { insertedCount, updatedCount }
});
```

4.6 HMAC-SHA256 Signed QR Code Engine

Student QR codes encode cryptographically signed strings to eliminate forged identity badges:

Generation Formula

$$\text{Signature} = \text{HMAC-SHA256}(\text{Unique_ID}, \text{QR_SECRET})$$
$$\text{QR_Payload} = \text{Unique_ID} + \text{"."} + \text{Signature}$$

Verification Algorithm (`POST /api/qr/verify`)

- 1. Split payload by delimiter `..`
- 2. Compute expected HMAC of `Unique_ID` using `QR_SECRET`.

3. Compare provided signature with expected signature using constant-time string comparison (`timingSafeEqual`) to prevent timing attacks.

4. If valid, return authenticated `student` record.

4.7 Sunday School Curriculum, Grouped Subjects & Teacher Assignments

Subjects are managed hierarchically per grade and academic year:

Orthodox Curriculum Subjects

- ከርዓተ ቤተክርስቲያን (Church Liturgy & Order)
- ነገረ ሃይማኖት (Dogmatics & Theology)
- የመጽሐፍ ቅዱስ ጥናት (Biblical Studies)
- ክርስቲያናዊ ስነ ምግባር (Christian Ethics)
- የቤተክርስቲያን ታሪክ (Church History)
- ግዕዝ ቋንቋ (Ge'ez Language)
- መዝሙር (Hymnody & Chants)
- በገና (BeGena Instrument)

Grouped Storage (SubjectGroup)

Instead of fragmented documents, subjects are stored in clean grade-grouped documents:

```
{
  "_id": "67b9...",
  "academicYear": "2018",
  "grade": "አንደኛ ክፍል",
  "gradeNumber": 1,
  "subjects": ["ከርዓተ ቤተክርስቲያን", "ነገረ ሃይማኖት", "የመጽሐፍ ቅዱስ ጥናት"]
}
```

4.8 Student Academic Results & University Letter Grading

The student grading module records continuous assessments and final exams:

$$\text{\$}\text{\text{Total Score}} = \text{\text{Assignment 1 (10)}} + \text{\text{Assignment 2 (10)}} + \text{\text{Mid Test (30)}} + \text{\text{Final Exam (50)}}\text{\$}$$

Letter Grade Scale

Score Range	Grade	Description
90 – 100	A+	Excellent / እጅግ በጣም ከፍተኛ
85 – 89	A	Excellent
80 – 84	A-	Very Good
75 – 79	B+	Very Good
70 – 74	B	Good
65 – 69	B-	Good
60 – 64	C+	Satisfactory
50 – 59	C	Pass
45 – 49	D	Conditional Pass
0 – 44	F	Fail / የወደቀ

4.9 13-Month Ethiopian Fee Payment Tracking

Payments are tracked on an academic-year basis with lazy initialization across all 13 Ethiopian months:

```
{
  "studentId": "67b9...",
  "year": "2018",
  "data": {
    "Meskerem": { "status": "Paid", "amount": "500" },
    "Tikimt": { "status": "Paid", "amount": "500" },
    "Hidar": { "status": "Paid", "amount": "500" },
    "Tahsas": { "status": "Not Paid", "amount": "" },
  }
}
```

```
"Tir":      { "status": "Not Paid", "amount": "" },
"Yekatit": { "status": "Not Paid", "amount": "" },
"Megabit":  { "status": "Not Paid", "amount": "" },
"Miyazia":  { "status": "Not Paid", "amount": "" },
"Ginbot":   { "status": "Not Paid", "amount": "" },
"Sene":     { "status": "Not Paid", "amount": "" },
"Hamle":    { "status": "Not Paid", "amount": "" },
"Nehase":   { "status": "Not Paid", "amount": "" },
"Pagumē":   { "status": "Not Paid", "amount": "" }
}
}
```

4.10 Student Registration Requests & Approval Workflow

When a facilitator registers a student without direct admin write permissions:

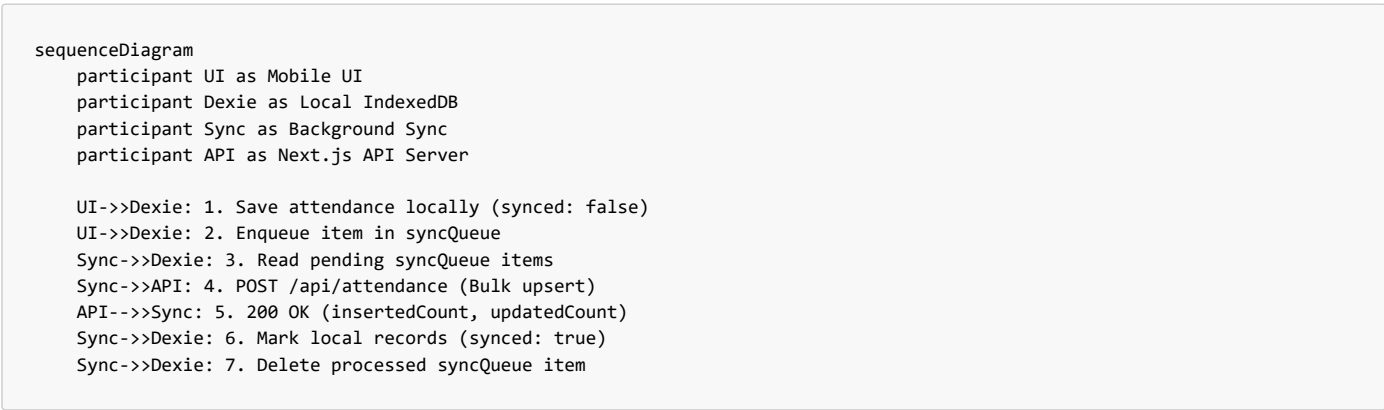
1. Facilitator submits request to `POST /api/student-requests` with full `studentData` (`status: "pending"`).
2. Super Admin views pending requests at `/admin/student-requests`.
3. Upon review:

○ **Approve:** Super Admin sends `PATCH /api/student-requests` with `status: "approved"`. The server automatically inserts the student into the main `students` collection and generates the HMAC QR code.

○ **Reject:** Super Admin provides `status: "rejected"` with `rejectionReason` (e.g. "መረጃው ያልተሟላ ነው").

4.11 Offline-First Mobile Architecture & Dexie Sync Queue

The Capacitor mobile client uses [Dexie.js](#) for local IndexedDB persistence:



5. Complete API Route Directory (38 Endpoints)

Method	Endpoint	Description	Auth Required	Request / Query Body	Response Status
GET	<code>/api/auth/csrf</code>	Returns CSRF token for web login	None	None	200 { <code>csrfToken</code> }
POST	<code>/api/auth/callback/credentials</code>	Web credentials authentication	None	URL-encoded <code>csrfToken</code> , <code>email</code> , <code>password</code>	200 / 302
GET	<code>/api/auth/session</code>	Get active session profile	Session Cookie	None	200 { <code>user</code> }
POST	<code>/api/auth/signout</code>	Invalidate web session	Session Cookie	None	200 / 302
POST	<code>/api/mobile/auth</code>	Mobile app JWT authentication	None	{ <code>email</code> , <code>password</code> }	200 { <code>token</code> , <code>user</code> } / 401
OPTIONS	<code>/api/mobile/auth</code>	CORS preflight for mobile auth	None	None	204 No Content
POST	<code>/api/setup/super-admin</code>	Bootstrap initial Super Admin	None (one-time)	{ <code>name</code> , <code>email</code> , <code>password</code> , <code>confirm</code> }	201 Created / 200 (exists)
GET	<code>/api/students</code>	List students with filters	Session Cookie	? <code>grade</code> =& <code>academicYear</code> =& <code>sex</code> =& <code>limit</code> =	200 <code>Student[]</code>

Method	Endpoint	Description	Auth Required	Request / Query Body	Response Status
POST	/api/students	Create student record	Admin / Facilitator	Student JSON payload	201 { _id } / 403
DELETE	/api/students	Delete student record	Admin	{ id: "studentId" }	200 { message }
GET	/api/students/[studentId]	Get student by MongoDB ID / Unique_ID	Session Cookie	Path param studentId	200 Student / 404
PUT	/api/students/[studentId]	Full student profile update	Admin	Updated Student fields	200 Student
PATCH	/api/students/[studentId]	Update student photo or regenerate QR	Session Cookie	{ photo_data_url } or { generateQR: true }	200 Student
POST	/api/students/check-duplicate	Check name+sex duplicate	Session Cookie	{ First_Name, Father_Name, Grandfather_Name, Mothers_Name, Sex }	200 { exists: boolean }
POST	/api/students/count	Count students in grade	Session Cookie	{ academicYear, grade, classification }	200 { count }
GET	/api/students/total	Get total student count	Session Cookie	None	200 { total: number }
GET	/api/students/[studentId]/qr	Stream student QR PNG image	None / Session	Path param studentId	200 image/png
GET	/api/students/[studentId]/payments	Legacy student payments lookup	Session Cookie	Path param studentId	200 Payment[]
GET	/api/attendance	Query attendance records & summaries	Session Cookie	? date=&grade=&academicYear=&summary=true	200 Attendance[] or { total, present }
POST	/api/attendance	Bulk upsert attendance	Session Cookie	{ date, attendance: Attendance[] }	200 { success, insertedCount, updatedCount }
GET	/api/attendance/[studentId]	Student attendance history	Session Cookie	Path param studentId	200 Attendance[]
GET	/api/attendance/temp	Get draft / uncommitted attendance	Session Cookie	?date=&markedBy=	200 TempAttendance[]
GET	/api/facilitators	List facilitators (scoped)	Super Admin / HR	None	200 Facilitator[]
POST	/api/facilitators	Create Attendance Facilitator	Super Admin / HR	{ name, email, password, role, grade }	201 { _id } / 409
PUT	/api/facilitators	Update facilitator profile	Super Admin / HR	{ id, name, email, role, grade }	200 Facilitator
DELETE	/api/facilitators	Delete facilitator	Super Admin / HR	{ id }	200 { message }
GET	/api/facilitators/roles	List facilitator role constants	None / Session	None	200 [{ value, label }]
PATCH	/api/facilitators/[id]	Toggle canAddStudent permission	Super Admin / HR	{ canAddStudent: boolean }	200 Facilitator

Method	Endpoint	Description	Auth Required	Request / Query Body	Response Status
GET	/api/facilitators/total	Total facilitators count	Session Cookie	None	200 { total: number }
GET	/api/education-facilitators	List Education Facilitators	Education Admin	None	200 User[]
POST	/api/education-facilitators	Create Education Facilitator	Education Admin	{ name, email, password }	201 { _id }
PUT	/api/education-facilitators	Update Education Facilitator	Education Admin	{ id, name, email }	200 User
DELETE	/api/education-facilitators	Delete Education Facilitator	Education Admin	{ id }	200 { message }
GET	/api/teachers	List Teachers	Education Admin	None	200 User[]
POST	/api/teachers	Create Teacher profile	Education Admin	{ name, email, grade, assignedSubjects }	201 { _id }
PUT	/api/teachers	Update Teacher profile	Education Admin	{ id, name, email, grade, assignedSubjects }	200 User
DELETE	/api/teachers	Delete Teacher profile	Education Admin	{ id }	200 { message }
GET	/api/subjects	List grouped subjects	Session Cookie	?academicYear=&grade=	200 SubjectGroup[]
POST	/api/subjects	Add subject to grade group	Education Admin	{ name, grade, gradeNumber, academicYear }	201 SubjectGroup
POST	/api/subjects/batch	Batch create multiple subjects	Education Admin	{ subjects: SubjectInput[] }	200 { results, summary }
DELETE	/api/subjects/[id]	Remove subject from group	Education Admin	{ name: "subjectName" }	200 SubjectGroup
GET	/api/teacher-assignments	List teacher-subject mappings	Session Cookie	None	200 Assignment[]
POST	/api/teacher-assignments	Create teacher assignment	Education Admin	{ teacherId, subjectId, grade, academicYear }	201 { _id }
DELETE	/api/teacher-assignments/[id]	Delete teacher assignment	Education Admin	Path param id	200 { message }
GET	/api/category-periods	List category periods	Session Cookie	?academicYear=	200 CategoryPeriod[]
POST	/api/category-periods	Upsert category period	Super Admin	CategoryPeriod JSON	200 CategoryPeriod
GET	/api/student-results	List all academic results	Session Cookie	?studentId=	200 StudentResult[]
POST	/api/student-results	Record student exam results	Education Admin	StudentResult JSON	201 StudentResult
GET	/api/student-results/[id]	Get student result by ID	Session Cookie	Path param id	200 StudentResult
PUT	/api/student-results/[id]	Update student result	Education Admin	Updated result fields	200 StudentResult
DELETE	/api/student-results/[id]	Delete student result	Education Admin	Path param id	200 { message }
GET	/api/student-requests	List student requests	Session Cookie	?status=pending	200 StudentRequest[]

Method	Endpoint	Description	Auth Required	Request / Query Body	Response Status
GET	/api/student-requests/[id]	Get student request by ID	Session Cookie	Path param id	200 StudentRequest
POST	/api/student-requests	Submit student registration request	Facilitator	{ studentData, requestedBy, requestedByName }	201 { _id }
PATCH	/api/student-requests	Approve or reject student request	Super Admin	{ id, status, approvedBy, rejectionReason }	200 StudentRequest
GET	/api/payment	13-month payment status	Session Cookie	?year=&studentId=	200 { year, studentId, data }
POST	/api/payment	Save monthly payment status	Session Cookie	{ year, studentId, data }	200 { message }
GET	/api/admin-users	List department admins	Super Admin	None	200 User[]
POST	/api/admin-users	Create HR or Education Admin	Super Admin	{ name, email, password, role }	201 { _id }
PUT	/api/admin-users	Update department admin	Super Admin	{ id, name, email }	200 User
DELETE	/api/admin-users	Delete department admin	Super Admin	{ id }	200 { message }
POST	/api/qr/verify	Verify signed HMAC QR payload	None / Session	{ text: "uniqueId.hmacSignature" }	200 { student } / 400
GET	/api/sheet/health	Google Sheets health check	Session Cookie	None	200 { status: "ok" }
GET	/api/sheet	Fetch Google Sheets rows	Session Cookie	?sheet=students	200 any[]
GET	/api/cron/aggregate-attendance	Aggregate daily attendance statistics	Internal / Admin	?date=2018-05-15	200 { success, aggregated }

6. Database Schema & MongoDB Collections

1. users

```
{
  _id: ObjectId,
  name: string,
  email: string,           // Case-insensitive indexed unique string
  password: string,       // Bcrypt hash ($2a$10$...)
  role: "Super Admin" | "HR Admin" | "Education Admin" | "Attendance Facilitator" | "Education Facilitator" | "Teacher",
  grade?: string | string[], // Assigned grades for facilitators / teachers
  assignedSubjects?: string[],
  canAddStudent?: boolean, // Dynamic student creation permission
  createdAt: string,
  updatedAt?: string
}
```

2. students

```
{
  _id: ObjectId,
  Unique_ID: string,      // Unique index e.g. "BH-2018-001"
  First_Name: string,     // Amharic First Name
}
```

```

Father_Name: string,      // Amharic Father Name
Grandfather_Name: string, // Amharic Grandfather Name
Mothers_Name: string,     // Mother's Full Name
Christian_Name: string,   // Baptismal Name (የክርስቲና ስም)
photo_data_url?: string,  // Base64 PNG/JPEG Data URL
DOB_Date: string,
DOB_Month: string,
DOB_Year: string,
Age: number,
Sex: "Male" | "Female",
Phone_Number: string,
Class: string,
Occupation: string,
School?: string,
Address: string,
Academic_Year: string,    // e.g. "2018"
Grade: string,            // e.g. "አንደኛ ክፍል"
Classification: "Regular" | "Extension" | "SignLanguage" | "Summer" | "begen",
qr_code?: string         // Base64 signed QR Data URL
}

```

3. attendance

```

{
  _id: ObjectId,
  studentId: string,      // Reference to students._id
  date: string,           // Ethiopian date "YYYY-MM-DD"
  present: boolean,
  hasPermission: boolean,
  reason?: string,
  markedBy?: string,
  timestamp?: string
}

```

Compound Index: { studentId: 1, date: 1 } (Unique).

4. subjects (Grouped Subject Schema)

```

{
  _id: ObjectId,
  academicYear: string,   // e.g. "2018"
  grade: string,          // e.g. "አንደኛ ክፍል"
  gradeNumber: number,    // e.g. 1
  subjects: string[]      // ["ከርዓተ ቤተክርስቲያን", "ነገረ ሃይማኖት", ...]
}

```

5. student_results

```

{
  _id: ObjectId,
  studentId: string,
  studentName: string,
  subjectId: string,
  subjectName: string,
  academicYear: string,
  assignment1: number,    // 0 - 10
  assignment2: number,    // 0 - 10
  midTest: number,       // 0 - 30
  finalExam: number,     // 0 - 50
  totalScore: number,     // 0 - 100
  average: number,
  grade: string,          // "A+", "A", "B", ...
  remarks?: string,
  recordedDate: string
}

```

6. payment_status

```
{
  _id: ObjectId,
  studentId: string,
  year: string,
  data: {
    Meskerem: { status: "Paid" | "Not Paid", amount: string },
    Tikimt: { status: "Paid" | "Not Paid", amount: string },
    Hidar: { status: "Paid" | "Not Paid", amount: string },
    Tahsas: { status: "Paid" | "Not Paid", amount: string },
    Tir: { status: "Paid" | "Not Paid", amount: string },
    Yekatit: { status: "Paid" | "Not Paid", amount: string },
    Megabit: { status: "Paid" | "Not Paid", amount: string },
    Miyazia: { status: "Paid" | "Not Paid", amount: string },
    Ginbot: { status: "Paid" | "Not Paid", amount: string },
    Sene: { status: "Paid" | "Not Paid", amount: string },
    Hamle: { status: "Paid" | "Not Paid", amount: string },
    Nehase: { status: "Paid" | "Not Paid", amount: string },
    Pagumē: { status: "Paid" | "Not Paid", amount: string }
  }
}
```

7. category_periods

```
{
  _id: ObjectId,
  classification: "Regular" | "Extension" | "SignLanguage" | "Summer" | "begenā",
  academicYear: string,
  startDate: string,
  endDate: string,
  registrationClosedDate: string,
  isActive: boolean
}
```

8. student_requests

```
{
  _id: ObjectId,
  studentData: Student,
  requestedBy: string,
  requestedByName: string,
  status: "pending" | "approved" | "rejected",
  createdAt: Date,
  updatedAt: Date,
  approvedBy?: string,
  rejectionReason?: string
}
```

7. Automated Testing, QA & Tooling

The codebase includes an end-to-end automated QA testing suite located at [scripts/run-automation-tests.mjs](file:///c:/Users/Ym/Desktop/b/ss/Birhane-Hiwot-sunday-school-attendance/scripts/run-automation-tests.mjs).

Test Suite Structure (63 Test Cases)

1. Suite 1: Authentication & Setup:

- AUTH-012: Super Admin Bootstrap & Verification
- AUTH-013: Invalid confirmation key rejection
- MOB-001 to MOB-003: Mobile JWT Authentication, invalid credentials & CORS preflight
- AUTH-011, AUTH-001, AUTH-010: CSRF token, NextAuth credentials login & session cookie verification

2. Suite 2: Facilitators & Roles:

- FAC-011: Role constant lookups
- FAC-003, FAC-004, FAC-001, FAC-010: Attendance Facilitator CRUD & validation
- FAC-009: Dynamic canAddStudent permission toggling
- FAC-012: Education Facilitator CRUD

3. Suite 3: Teachers, Subjects & Curriculum:

- TCH-001, TCH-002: Teacher profiles & assignments
- SUB-001, SUB-002, SUB-004: Grouped subjects, additions, & batch creation
- TAS-001, TAS-002: Teacher assignments

4. Suite 4: Category Registration Periods:
- PER-001, PER-003, PER-004: List, upsert, and validation
5. Suite 5: Student Management:
- STU-006, STU-007: Real Amharic student registration

STU-020, STU-021: 5-tuple duplicate name checks

STU-001, STU-004, STU-005: List, query lookup, path lookup

STU-023, STU-025: Grade count & total count

STU-015, STU-017, STU-018: Profile PUT, photo PATCH, QR regeneration
6. Suite 6: Attendance Operations:
- ATT-006, ATT-009: Bulk upsert & idempotency

ATT-001, ATT-003, ATT-012: Date queries, summary mode & student history
7. Suite 7: Academic Results & Grading:
- RES-003: Assessment scoring & Letter Grade calculation (A+ to F)

RES-001, RES-002, RES-010, RES-011: Query, update, and recalculation
8. Suite 8: 13-Month Payments:
- PAY-001, PAY-003: Lazy 13-month initialization & updates
9. Suite 9: Registration Requests:
- REQ-001, REQ-003, REQ-004, REQ-005: Request lifecycle (submit, query, approve)
10. Suite 10: QR & Utilities:
- QR-005: Empty payload rejection

UTIL-001, UTIL-002: Google Sheets health check & attendance aggregation cron
11. Teardown:
- TEARDOWN-001 to TEARDOWN-006: Automated cleanup of test students, results, teachers, and facilitators.

Running Automated Tests

```
npm run test
# or
npm run test:api
```

8. Scripts, Migrations & Excel Seeders

Located in [scripts/](file:///c:/Users/Ym/Desktop/b/ss/Birhane-Hiwot-sunday-school-attendance/scripts/):

Script	Command	Purpose
run-automation-tests.mjs	npm run test	Full end-to-end API test runner with session cookie jar management.
seed-from-excel.mjs	npm run seed:excel	Parses the official Ethiopian Sunday School Excel roster (docs/የ2018 ዓ.ም የክሰዓት መርሃ ግብር...xlsx) and seeds MongoDB. Supports --dry-run.
migrate-subjects-to-grouped.mjs	node scripts/migrate-subjects-to-grouped.mjs	Migrates legacy individual subject records into grouped SubjectGroup collections.
init-indexes.mjs	node scripts/init-indexes.mjs	Ensures all unique and compound database indexes exist in MongoDB.
inspect-grades.mjs	node scripts/inspect-grades.mjs	Audits grade string normalization and classification consistency.

9. Configuration & Environment Variables

All secrets and configuration keys are defined in .env.local:

```
# MongoDB Connection
MONGODB_URI=mongodb://ym:sunday_school@ac-zoyajqc-shard-00-00.tchv8s6.mongodb.net:27017,ac-zoyajqc-shard-00-01.tchv8s6.mongodb.net:27017,ac-zoyajqc-shard-00-02.tchv8s6.mongodb.net:27017/?ssl=true&replicaSet=atlas-cbb0fr-shard-0&authSource=admin&retryWrites=true&w=majority&appName=Cluster0
MONGODB_DB=sunday_school

# NextAuth Web Authentication
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=fQBg7LV2yoNtuKwAZItwZAZTZAKby1W8

# Cryptographic QR Code Secret
QR_SECRET=fa53e51e20ee201b0f5a5efbbbb5d5851c49a0b711b992ffaddeaaafa015aa690

# Google Sheets Integration (Optional)
```

```
GOOGLE_SERVICE_ACCOUNT_TYPE=service_account
GOOGLE_SHEET_ID=1kZZXZrpBTK9aaZ5zckkLLh2vgG6Amy0MAn09Zyj9n0
GOOGLE_SERVICE_ACCOUNT_PROJECT_ID=sunday-school-469020
GOOGLE_SERVICE_ACCOUNT_PRIVATE_KEY_ID=dd1023c4a76db2bee6b686532eb831b5346df098
GOOGLE_SERVICE_ACCOUNT_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n..."
GOOGLE_SERVICE_ACCOUNT_CLIENT_EMAIL=sunday-school@sunday-school-469020.iam.gserviceaccount.com

# Redis Distributed Locking (Optional Upstash URL / Local In-Memory Fallback)
REDIS_URL=https://...
REDIS_TOKEN=...
```

10. Local Development & Deployment Guide

Prerequisites

- Node.js **v20.x** or **v22.x**
- npm **v10+** or **v11+**
- MongoDB Atlas active cluster or local MongoDB instance

Installation & Startup

```
# 1. Clone repository and install dependencies
npm install

# 2. Start the development server (with Turbopack)
npm run dev

# 3. Access the web interface
open http://localhost:3000

# 4. View interactive Swagger API documentation
open http://localhost:3000/api-docs

# 5. Run the automated test suite
npm run test
```

Initial Super Admin Bootstrap

On a fresh deployment, initialize the primary Super Admin account:

```
curl -X POST http://localhost:3000/api/setup/super-admin \
-H "Content-Type: application/json" \
-d '{
  "name": "ለቀ ትጉሃን ገብረመስቀል",
  "email": "superadmin@birhanehiwot.org",
  "password": "SuperAdminPass123!",
  "confirm": "CREATE_SUPER_ADMIN"
}'
```

Production Build & Deployment (Vercel)

```
npm run build
npm run start
```

The project is configured with **vercel.json** and optimized for Vercel Edge/Serverless deployments with MongoDB connection pooling.